

Product Design Document

Team: **Squirtle Squad**

Team Members: Hoang Bui, Andre Phanavanh, Austin Ridge, Muhammad Taha

Team Lead: Andre Phanavanh

Centsible

Personal Finance Tracker

Revision Table

<i>Revision Number</i>	<i>Revision Date</i>	<i>Summary of Changes</i>	<i>Author(s)</i>
<i>0.1</i>	<i>06/07/2026</i>	Creation of Design Document 1	Squirtle Squad
<i>0.2</i>	<i>06/14/2026</i>		Squirtle Squad

Class Diagram(s)

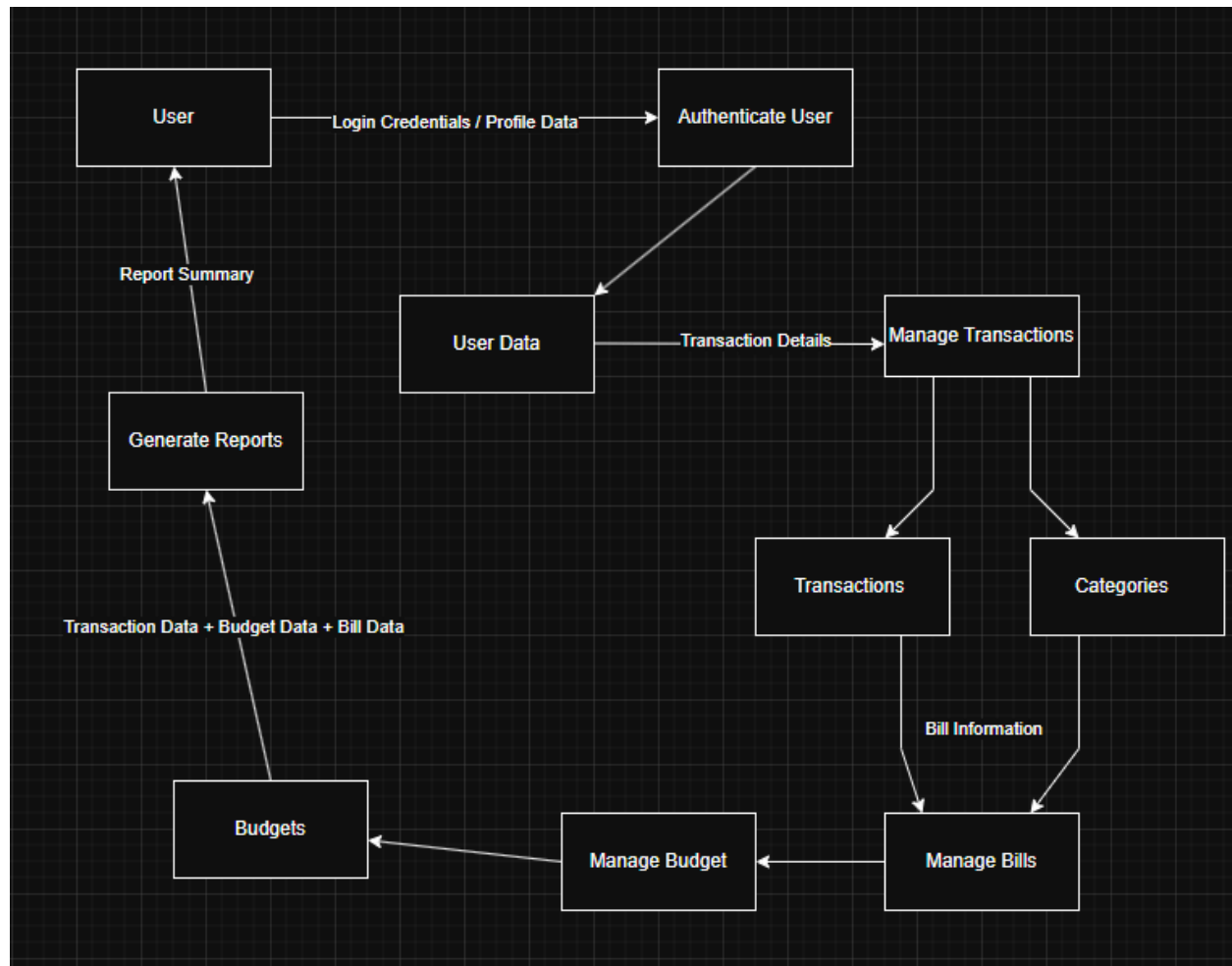
1. DFD - Data Flow Diagram
2. ERD - Entity-Relationship Diagram (Taha)
3. STD - State Transition Diagram
4. Information Architecture Diagram
5. And any other necessary diagrams
6. User Interface Wireframe/Screenshots
7. Prototype

Design Summary

This Data Flow Diagram (DFD) illustrates how information moves throughout the Centsible application. The user interacts with the system by authentication, managing transactions, tracking bills, setting budgets, and generating reports. Data is stored in the User, Transactions, Bill, Budgets, and Categories data stores. The Generate Report process combines financial information from multiple data stores to provide summary reports and insights for the user.

Design Rationale

This design was created to show how data flows between the user, system processes and data stores. Separating the system into authentication, Transaction management, bill management, budget management, and reporting processes improves organization and maintainability. The data stores reflects the main entities identified in the ERD and rapport the applications's core functionality. This DFD helps visualize how information is processed and shared throughout the system while maintaining a clear structure.

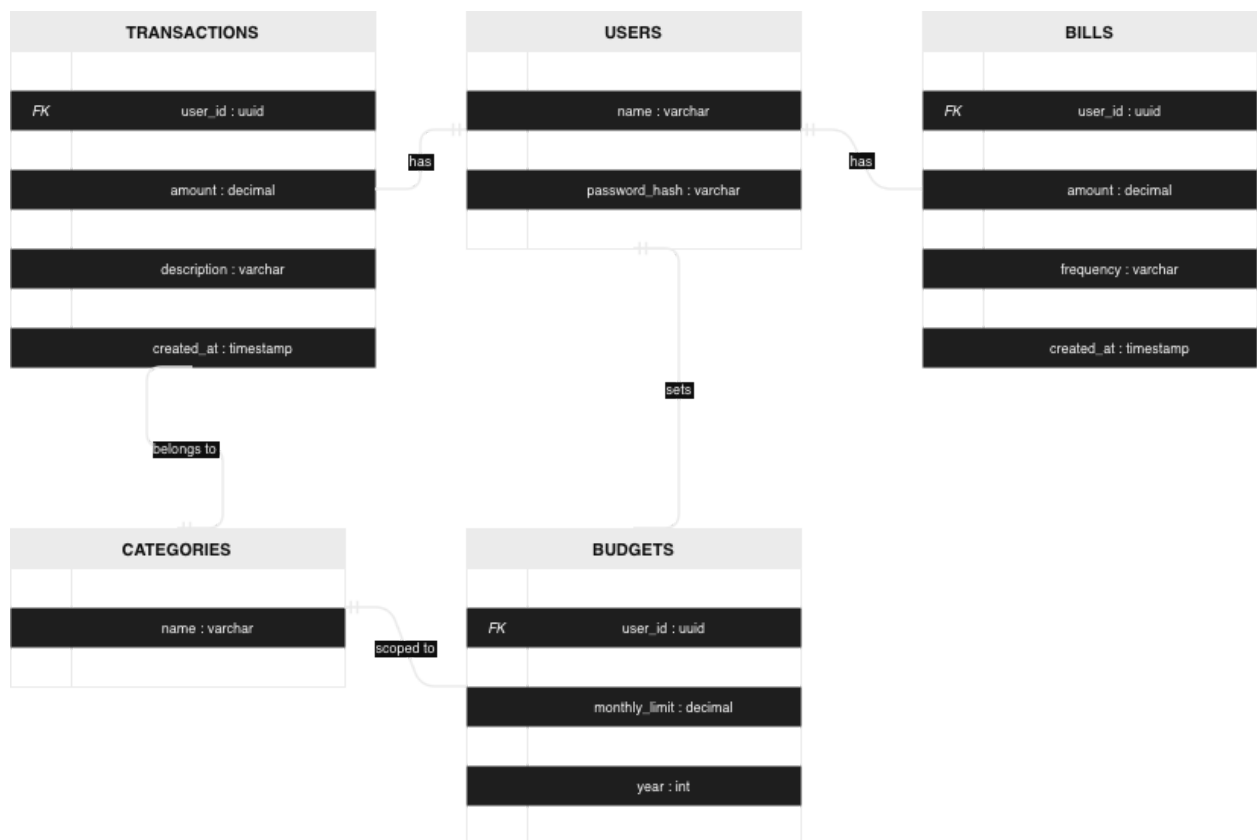


Design Summary

This entity relationship diagram shows the database structure of the Centsible application. The main table is Users because the system is centered around a registered user. A user can have transactions, bills, and budgets. Transactions store income and expense records, while bills store recurring payments and due dates. Budgets store monthly spending limits, and categories help organize transactions and budgets by type.

Design Rationale

The design makes sense because each part of the app needs to connect back to the logged-in user. The user_id foreign key connects users to their transactions, bills, and budgets. The category_id foreign key connects transactions and budgets to categories, which helps organize spending. This structure keeps the data clean and makes it easier for the system to show the dashboard, track expenses, manage bills, and calculate budget progress.

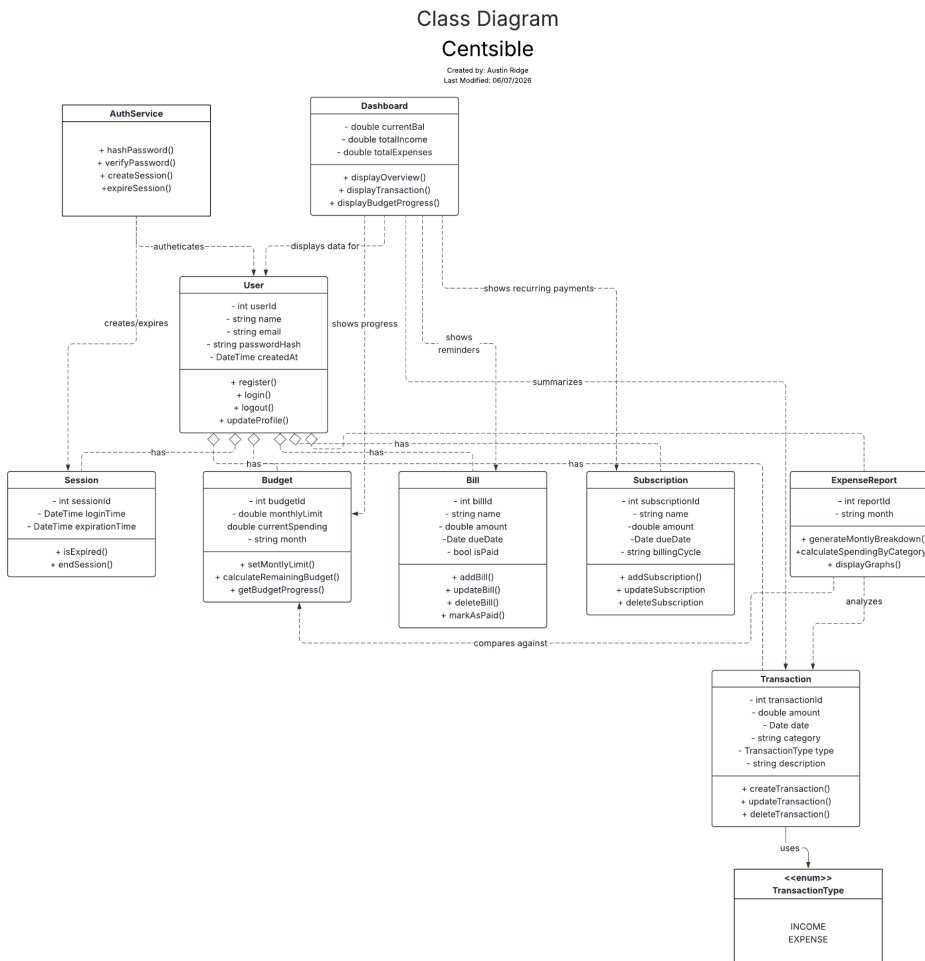


Design Summary

This class diagram shows the object-oriented design. The main class is User because most of the system's data belongs to a registered user. A user can have transactions, a budget, bills, subscriptions, sessions, and expense reports. The Transaction class stores income and expense records, and it uses the TransactionType enumeration to separate income from expenses. The Budget, Bill, and Subscription classes represent the main financial tracking features of the application. The Session class handles login sessions, while AuthService handles authentication functions. The Dashboard and ExpenseReport classes are included to show how the system displays summaries and reports based on the user's financial data.

Design Rationale

The reason for this design is to keep each part of the system separated by responsibility. The user acts as the central class since the application is based around a registered user managing their own financial information. The finance-related classes are connected to User because they represent data that belongs to the user. AuthService is separate because authentication is more of a system service than something the User class should directly handle. The dashboard is also separate because it does not really own the data, but instead just pulls information from transactions, budgets, bills, and subscriptions to display an overview. This design makes the system easier to understand, easier to modify, and matches the main requirements of the app.

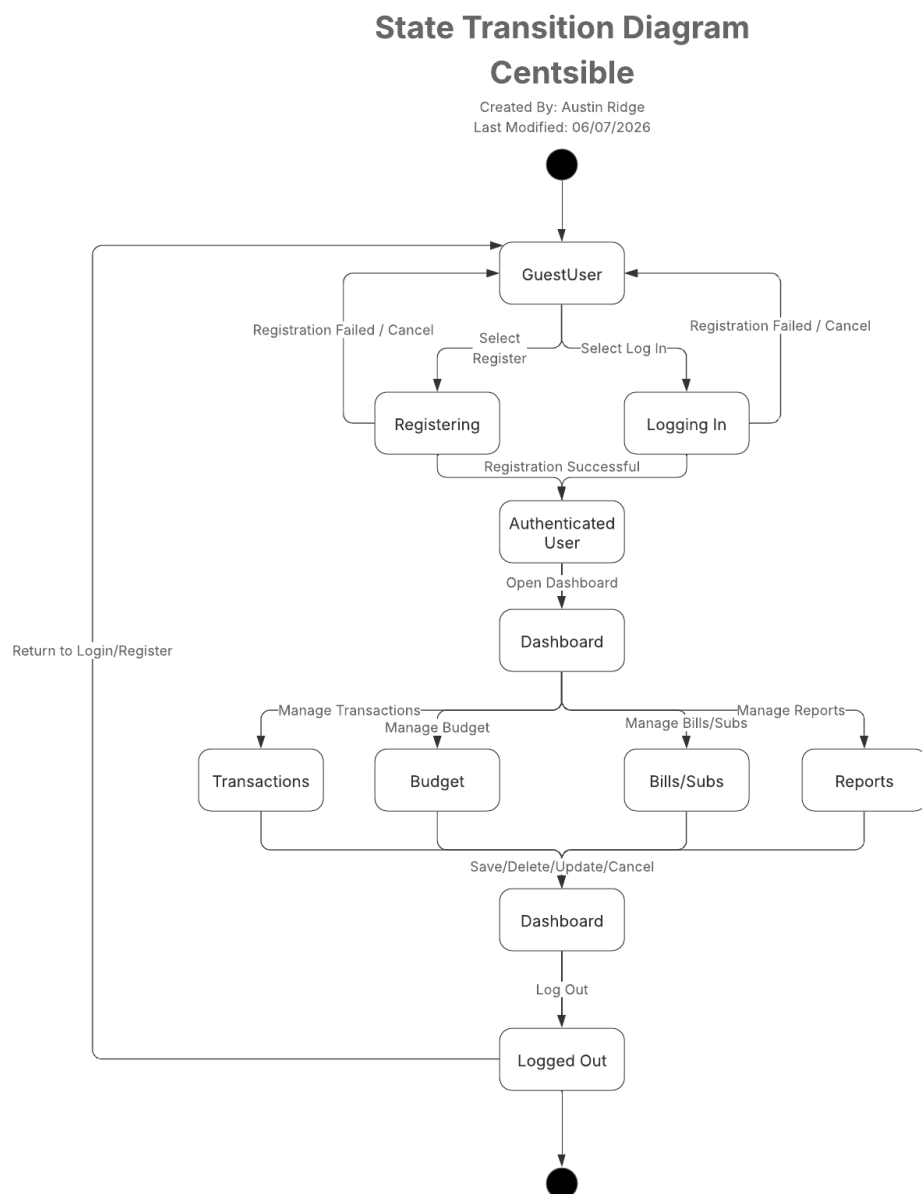


Design Summary

This state transition diagram shows the main flow of the Centsible application. A user starts as a guest, then either registers or logs in. If authentication is successful, the user becomes authenticated and can open the dashboard. From the dashboard, the user can manage transactions, budgets, bills/subscriptions, or reports, then return back to the dashboard after saving, deleting, updating, or canceling. The diagram also includes failure paths for registration/login and a logout state, which helps show how the system handles both normal use and basic account/session

Design Rationale

The design makes sense because the app is centered around a logged-in user moving between the main finance features. It also includes failure paths for registration/login and a logout state, which helps show how the system handles both normal use and basic account/session



behavior.

Design Summary

The User Interface Wireframe will consist of 5 main tabs: Dashboard, Transactions, Bills & Subscriptions, and Reports. The dashboard overview, which consists of all the general information about the user's finances. Transactions will list the user's wage/salary with the addition of expenses that have occurred. The Budget page will generate a suggested calculation of the user's monthly expenses based on their income. On the Bills & Subscriptions tab, this will list all the user's monthly billing cycles. Lastly, the Reports section will provide a summary of monthly income and spending and display it as categories through a chart visual.

Design Rationale

The layout was designed specifically to be simple for the user to navigate through. Having the information right in the center ensures that the most crucial information is the very first thing the user sees upon logging into the application. The dark color scheme was chosen because it allows the highlighting of the important information in a brighter color to be easily noticed.

